

What Your Cache Reveals About You

FRANCISCO GOUVEIA, Instituto Superior Técnico, University of Lisbon, Portugal

SEBASTIAN DAVID ALBU, Institute of Information Security (ISEC) / TU Graz, Austria

JOÃO MATOS, Instituto Superior Técnico, University of Lisbon, Portugal

SAUMYA ROY, Department of Biomedical Engineering and Health Systems - KTH Royal Institute of Technology, Sweden

SAMANEH SHAFEE, Department of Computer Science , University of Lisbon, Portugal

1 Introduction

Over the past decade, a series of high-impact microarchitectural attacks, such as *Spectre* [5], *Meltdown* [6], and *Rowhammer* [2], have demonstrated that, sometimes, performance-oriented processor features can leak information. These attacks revealed that even well-established hardware abstractions cannot be assumed to enforce strong security. Take, for example, the case of *Meltdown*, which targeted Intel processors since 1995, breaking the dogma that established hardware can be taken for granted from a security perspective.

Among the many forms these side channels take, cache-based attacks have emerged as one of the most powerful and extensively studied classes [8–10]. Modern processors expose a variety of behaviors that, while designed to improve performance, can inadvertently leak information across boundaries that should be properly isolated. By observing variations in access latency, one can infer which cache lines are being used by a process, thereby revealing aspects of the execution that were never intended to be observable [4, 7].

Cache template attacks, for instance, demonstrated that it is possible to profile arbitrary software and automatically identify memory accesses that correlate with high-level victim events [3]. This profiling phase produces templates that map cache activity to specific program states or actions, such as hovering over an application or initiating a video call. With this, an attacker can later monitor those states with high accuracy, being able to spy on the victim and their activities. These developments show that cache activity can indeed be measured and exploited at a level of granularity sufficient for cryptographic key extraction, but also for distinguishing behavioral and execution patterns in general-purpose applications.

Although this may seem inconsequential at first glance, this vulnerability can be exploited in malicious ways; the most intuitive would be blackmail. One can easily detect when a person is hovering over an app, recording their screen, or even participating in a voice call. With that, a malicious actor could use this information *à la* social engineering, to get sensitive information about a user.

This report investigates the feasibility of applying such techniques to detect user activity in common desktop software. Specifically, we explore whether profiling cache hits is sufficient to distinguish between different operational states of these applications. Unlike classical side-channel targets, these applications do not perform computations that depend on secrets. Nevertheless, they

exhibit characteristic memory-access patterns, allowing an attacker to build templates for these distinct states and infer user behavior indirectly through cache activity alone.

2 Attack Workflow

Our attack follows the structure of Cache Template Attacks (CTA) [3], but uses a simplified approach, illustrated in Figure 1. The original CTA framework [1] provides the primitives for flush-and-reload measurements, timing collection, and noise reduction, and is designed to build multi-line templates that map victim activity to sets of memory locations. However, in contrast to the full CTA methodology, our work adopts a minimalist variant that relies on a single memory address rather than a full template. During profiling, we identify an address whose cache behaviour shows a consistent correlation with specific victim operations, and this address becomes the sole target of our online probing phase.

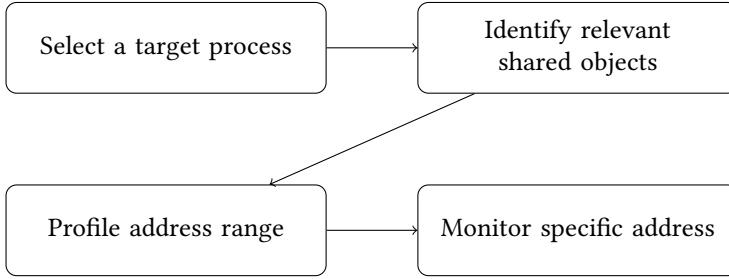


Fig. 1. Overview of the exploitation phase, showing how cache hits and misses are collected, thresholded, and interpreted to classify the victim’s activity.

The workflow begins with an offline profiling phase. However, before taking any further action, the first step is to determine the cache hit/miss threshold of one’s system. To achieve this, we can resort to the calibration tool developed for this purpose, the `./calibration/calibration` binary. Running it yields a histogram of cache hits and misses, which we can then analyze to find out our system’s specific threshold, the point where there are no more cache hits and cache misses start to be present. Note that for this process to yield trustworthy results, it is highly recommended that the laptop remain plugged in during both the calibration and profiling activities.

With the machine threshold delineated, we can advance to the profiling phase. The first thing is to decide which process, let us call it $\mathcal{P}$, we want to profile/exploit; this could be a messaging app, a text editor, or any desktop application of our choice. With $\mathcal{P}$ decided, the next step is to find the address range to attack. For that, we can execute the command `cat /proc/P/maps | grep "r-x"`. By querying the `/proc` filesystem for the virtual memory layout of the running process $\mathcal{P}$, it yields the output of `maps` which lists all memory segments, including their access permissions and corresponding file mappings. By filtering the entries that are marked with `"r-x"`, we extract the executable code regions of the library. These regions contain instructions that are actively executed by the application and therefore constitute suitable candidates for *flush-and-reload* probing. However, even if we filter out the results to have the executable regions, we are still faced with dozens, if not hundreds, of shared objects to analyze. Of course, it is very time-consuming to get through all of these regions. As such, we form a hypothesis and further refine the results. Let us imagine that $\mathcal{P}$ is a process involving some video recording or transmission. In that case, it would be reasonable to filter out for a shared object of a video encoder, such as `x264`. The resulting address ranges of the shared object serve as input for the next step, the profiling step.

After narrowing down the executable regions of the target process $\mathcal{P}$, the next step is to evaluate which individual instructions leak useful information. The utilized framework [1] provides a profiling tool designed explicitly for this purpose, the `profiling/spy` binary. Rather than probing each address range manually, the profiler automatically flushes and reloads selected offsets while the victim performs controlled activities. This enables us to compare the offset behavior between different states. In this first stage of profiling, we define a set of victim states that the attack should be able to distinguish, for example, participating in a Discord call, recording the screen with OBS, or hovering over Thunderbird. For each of these states, it repeatedly probes the same list of offsets and records the resulting access latencies. This produces a timing trace for every offset, allowing us to compare their behavior across different workloads. Offsets that show no variation across states are discarded. Instead, we aim to find a single offset whose cache-hit pattern changes consistently when the victim switches activities (i.e., accepting a voice call). As we employ a simplified variant of Cache Template Attacks, the objective is not to build a complete template, but rather to isolate one, stable, reproductive, leakage source. Once such an offset is found, it is re-profiled to ensure that the behavior remains stable across repeated sessions and does not represent an outlier.

After identifying the leakage-relevant address in the profiling phase, the exploitation phase continuously monitors this address to infer the victim's activity. Using the `exploitation/spy` binary, the attacker repeatedly flushed the selected cache line, waits briefly, and measures the reload latency. Each iteration yields a binary hit/miss value indicating whether the victim accessed code mapped to that line of code. Different activities produce distinct patterns: regular hit bursts during a voice call on Discord, and mostly misses in the idle state. Because the chosen address was selected for its clear state-dependent behavior, simple hit-rate rules are sufficient to establish a correlation and thus classify the current victim activity. Effectively, the attacker converts a single cache line into a real-time activity detection device.

3 Experiments

To evaluate the practicality of cache template attacks in real-world environments, experiments were conducted targeting various applications. The objective was not to extract sensitive data, but rather to demonstrate that externally observable cache patterns can reliably reveal user activity.

3.1 Detecting Mouse Hovering Activity in Thunderbird

The first experiment targeted Mozilla Thunderbird. Many components of the Thunderbird interface are rendered and managed through `libxul.so`, an extensive, shared library also used by Firefox. An offset for this library was found whose execution correlates with GUI updates, including event handling and redrawing triggered by mouse movement. During the experiment, the user interacted with Thunderbird by moving the mouse cursor across different UI elements, such as message list entries and toolbar icons. The attacker process continuously probed the selected cache lines and logged the hit counts. The resulting trace is shown in Figure 2.

The activity appears as a series of bursts with varying intensity. These bursts align with periods of cursor motion, reflecting the increased frequency of event handling routines and rendering updates within the instrumented region of `libxul.so`. Even though the attacker learns nothing about message content or sensitive data, the occurrence and intensity of UI interaction are readily observable.

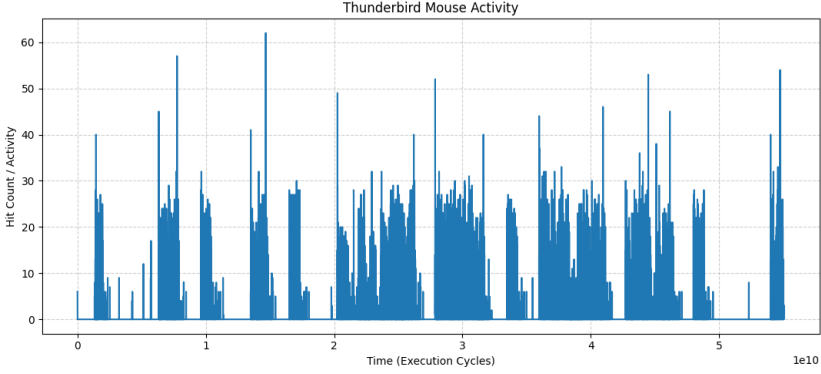


Fig. 2. Plot depicting cache hits, with spikes representing mouse activity on Thunderbird.

3.2 UI-Driven Cache Activity in Visual Studio Code

Building on the previous results, we turn our attention to Visual Studio Code (VS Code), whose Electron-based interface relies on a combination of shared libraries and custom rendering routines. We monitor a region within `libpango-1.0`, so that participates in VS Code's font rendering pipeline. During the capture window, the victim interacts with VS Code, opening new files and terminal windows.

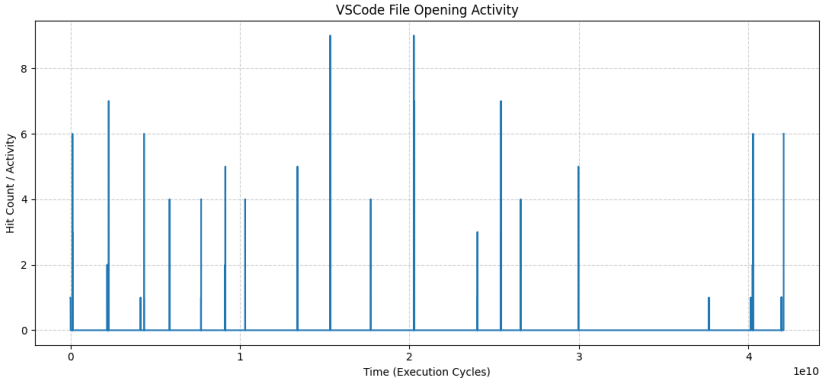


Fig. 3. Cache activity trace for VS Code. Short, periodic spikes correspond to the periods of text rendering.

As shown in Figure 3, the trace consists of a largely quiet baseline punctuated by clusters of short-lived spikes. Each spike aligns with the opening of a file or other action requiring text rendering. These signatures reveal that even routine interface interactions can be detected at fine granularity, enabling an attacker to infer when and how a user engages with the editor's components. Such visibility could be exploited in scenarios where user behavior within development environments is sensitive, allowing external observers to reconstruct interaction patterns without direct access to the application.

3.3 Browser Tab Activity in Mozilla Firefox

Given that Firefox shares the `libxul.so` library with Thunderbird, we extend our analysis to web browsing behaviors. The specific goal now is to detect the opening of new tabs, which is an action that triggers distinct memory allocation and rendering routines.

We monitor the offset `0x2fb2100` within `libxul.so`. The attack is executed with a 40-second timeout using the spy tool on the standard snap installation of Firefox. During the capture window, the victim opens a single new tab and subsequently leaves the browser in an idle state.

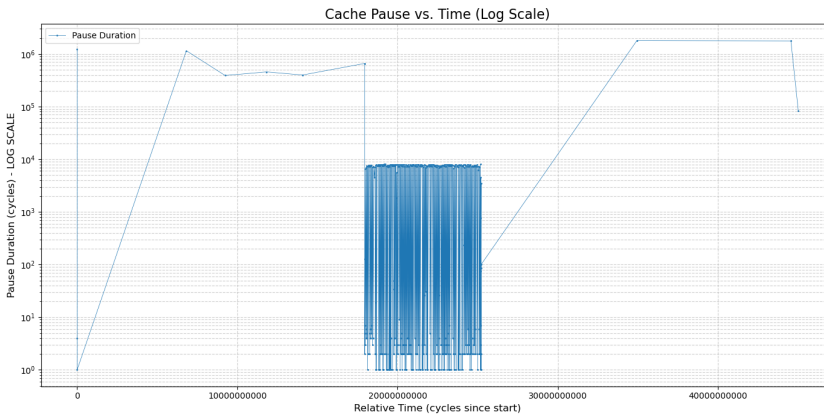


Fig. 4. Cache activity trace for Firefox. The prominent burst of traces corresponds to the user opening a single new tab, followed by a return to low-noise idle behavior.

The results, illustrated in Figure 4, show a near-silent baseline during the idle phase, interrupted by a massive, singular burst of activity in cache hits. This distinct signature enables an attacker to accurately count the number of tabs a user opens with high precision.

This capability poses specific risks in academic or professional monitoring contexts. For example, during proctored online examinations where a student is expected to remain on a single page, a side-channel attacker (or a malicious proctoring tool) could detect if the student opens new tabs to search for answers, even if the network traffic is encrypted or the screen is not being recorded.

3.4 Activity in the Tor Browser

Given that Tor Browser is based on Firefox, we extend our analysis to web browsing behaviors. The specific goal now is to detect any activity in the browser, such as clicks or opening of tabs, which are actions that trigger distinct memory allocation and rendering routines.

We monitor the offset `0x218b00` within `libgtk-3.so.0.2419.32`. The attack is executed with a 40-second timeout using the spy tool on the standard Flatpak installation of the Tor browser. During the capture window, the victim opens multiple tabs and performs activity within the browser.

The results, as illustrated in Figure 5, reveal a distinct pattern of activity characterized by a short burst of cache hits followed by an extended period of low-noise idle behavior. This signature provides a potential means for detecting usage within the browser, posing significant risks to individuals and organizations reliant on maintaining their privacy, particularly in high-security or high-risk situations where the consequences of unauthorized access could be severe.

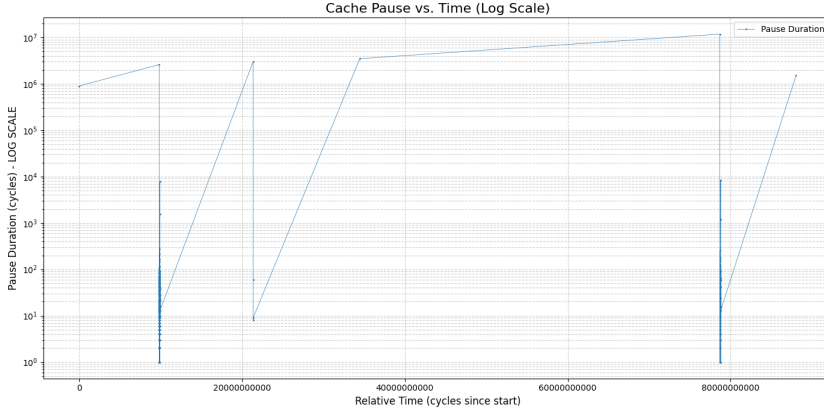


Fig. 5. Cache activity trace for Tor browser. The prominent burst of traces corresponds to the user doing any activity within the window, followed by a return to low-noise idle behavior.

3.5 Video Playing in VLC

One of the most commonly used software programs for playing videos is VLC. Knowing when a person is watching a video can be valuable information for an attacker. As such, by looking through the `iHD_drv_video.so` shared library for video drivers, we can indeed establish a correlation. Please note that this library is only present in `/proc/X/maps` if and only if a video is loaded in VLC. By profiling the offset `0x17be00`, we can distinctly observe that there is only activity when the video is playing. With that said, we can confidently establish the correlation between these observations and the victim's state of playing a video, as per the Figure 6, presented below.

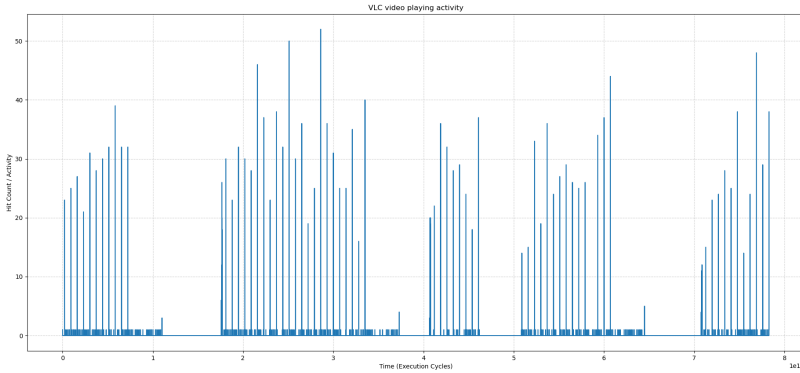


Fig. 6. Cache activity trace for VLC. The sustained activity clearly correlates with playing a video.

3.6 Recording Detection in OBS

Combining the findings above with a recording application, such as OBS, we have a powerful opportunity for blackmail. Let us imagine that we know someone was on Tor, or using any video

conference application (e.g., Discord). Then we could infer that the call was most likely inadvertently recorded. As OBS uses many shared libraries, it would be cumbersome to go through them all. With that said, the first hunch was to look for video encoders that would activate during a recording, such as x264, and it was indeed correct. After identifying the offset `0x3100` as a trustworthy indicator, the address was recorded for 40 seconds while the victim did some recordings in the meantime. The results, illustrated below in Figure 7, reveal a clear pattern of activity that only happens when a recording is underway. As in previous experiments, the attacker can confidently observe when a recording is being made.

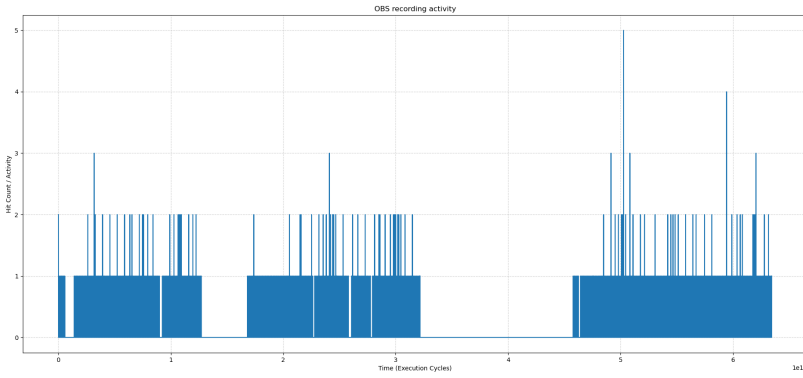


Fig. 7. Cache activity trace for OBS. The sustained activity clearly distinguishes the active recording state.

3.7 Voice Channel Detection in Discord

Finally, we investigate the proprietary communication platform Discord to determine if real-time voice activity can be inferred. Unlike the previous targets, this experiment directly probes the main Discord binary.

We identify offset `0x2287e00` as a high-confidence indicator of voice channel participation. The attacker monitored this address for 40 seconds while the victim joined a voice channel. The recorded trace demonstrates a sustained pattern of cache hits that correlates with the continuous audio processing and network transmission required by the voice protocol.

Figure 8 confirms that participation in a voice call creates a unique, observable footprint. This leakage has severe privacy implications regarding surveillance and social engineering. A malicious actor could use this information to stalk a victim, determining exactly when they are available or speaking to others. Furthermore, this data could be leveraged for blackmail. For instance, proving that an employee was on a Discord call during work hours, or contradicting a victim’s claim that they were offline.

4 Conclusion

This work demonstrates that microarchitectural side-channel attacks are not limited to the extraction of cryptographic keys but serve as a potent mechanism for inferring high-level user behavior in modern desktop applications. By adapting the Cache Template Attack methodology to a minimalist, single-address monitoring approach, we successfully correlated specific cache lines with distinct

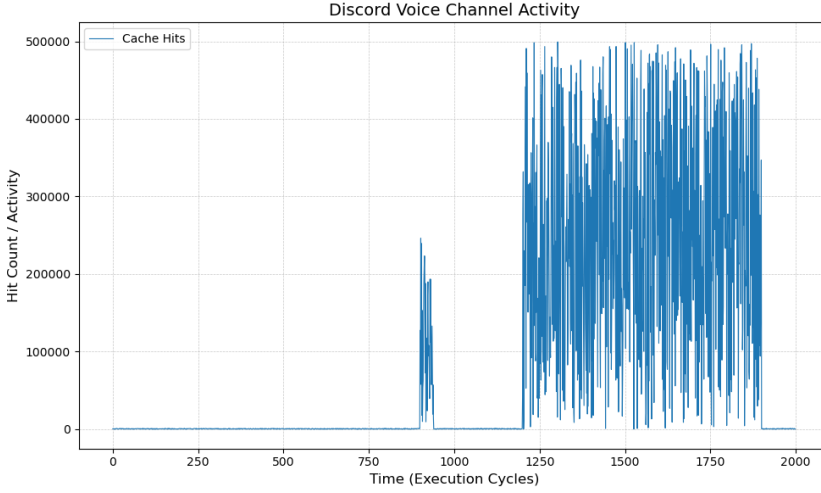


Fig. 8. Cache usage pattern during a Discord voice call. The sustained activity clearly distinguishes the active call state from the application’s background idle noise. The small spikes before the large activity illustrate typing in chat before joining a call.

operational states in OBS, VLC, Mozilla Thunderbird, as well as Mozilla Firefox, and Discord. Our experiments confirm that even without privileged access or complex multi-line templates, an attacker can discern sensitive activities, such as opening browser tabs or joining voice calls, with high temporal precision.

The implications of these findings extend into critical privacy domains. We highlighted how such leakage facilitates surveillance, social engineering, and academic dishonesty (e.g., detecting cheating during online exams). The ability to convert a shared library’s memory access pattern into a real-time activity log fundamentally undermines the assumption of isolation between user-space processes.

Future Work

While our current approach relies on the manual visual inspection of timing traces to identify activity bursts, the natural progression of this research is the development of an automated classification pipeline. Future implementations could introduce a secondary analysis tool designed to ingest raw spy traces in real-time.

For high-contrast events, such as the sustained activity observed in Discord voice calls, a simple sliding-window threshold mechanism would suffice to trigger alerts. However, for more nuanced patterns, such as distinguishing between mouse hovering and background rendering updates, we propose integrating a lightweight Neural Network (NN) or a Machine Learning classifier (e.g., Random Forest). Such a model could be trained on profiled datasets to automatically recognize and label user activities with high confidence, effectively turning the side-channel leakage into a fully autonomous surveillance feed.

References

- [1] Daniel Gruss et al. 2015. Cache Template Attacks. https://github.com/isec-tugraz/cache_template_attacks. Accessed: 2025-03-18.
- [2] Daniel Gruss, Cl  mentine Maurice, and Stefan Mangard. 2016. Rowhammer.js: A remote software-induced fault attack in javascript. In *International conference on detection of intrusions and malware, and vulnerability assessment*. Springer, 300–321.
- [3] Daniel Gruss, Raphael Spreitzer, and Stefan Mangard. 2015. Cache template attacks: Automating attacks on inclusive {Last-Level} caches. In *24th USENIX Security Symposium (USENIX Security 15)*. 897–912.
- [4] David Gullasch, Endre Bangerter, and Stephan Krenn. 2011. Cache games—bringing access-based cache attacks on AES to practice. In *2011 IEEE Symposium on Security and Privacy*. IEEE, 490–505.
- [5] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, et al. 2020. Spectre attacks: Exploiting speculative execution. *Commun. ACM* 63, 7 (2020), 93–101.
- [6] Moritz Lipp, Michael Schwarz, Daniel Gruss, Thomas Prescher, Werner Haas, Jann Horn, Stefan Mangard, Paul Kocher, Daniel Genkin, Yuval Yarom, et al. 2020. Meltdown: Reading kernel memory from user space. *Commun. ACM* 63, 6 (2020), 46–56.
- [7] Fangfei Liu, Yuval Yarom, Qian Ge, Gernot Heiser, and Ruby B Lee. 2015. Last-level cache side-channel attacks are practical. In *2015 IEEE symposium on security and privacy*. IEEE, 605–622.
- [8] Dag Arne Osvik, Adi Shamir, and Eran Tromer. 2006. Cache attacks and countermeasures: the case of AES. In *Cryptographers’ track at the RSA conference*. Springer, 1–20.
- [9] Fabian Rauscher, Carina Fiedler, Andreas Kogler, and Daniel Gruss. 2025. A systematic evaluation of novel and existing cache side channels. In *Network and Distributed System Security Symposium (NDSS) 2025*.
- [10] Yuval Yarom and Katrina Falkner. 2014. {FLUSH+ RELOAD}: A high resolution, low noise, l3 cache {Side-Channel} attack. In *23rd USENIX security symposium (USENIX security 14)*. 719–732.